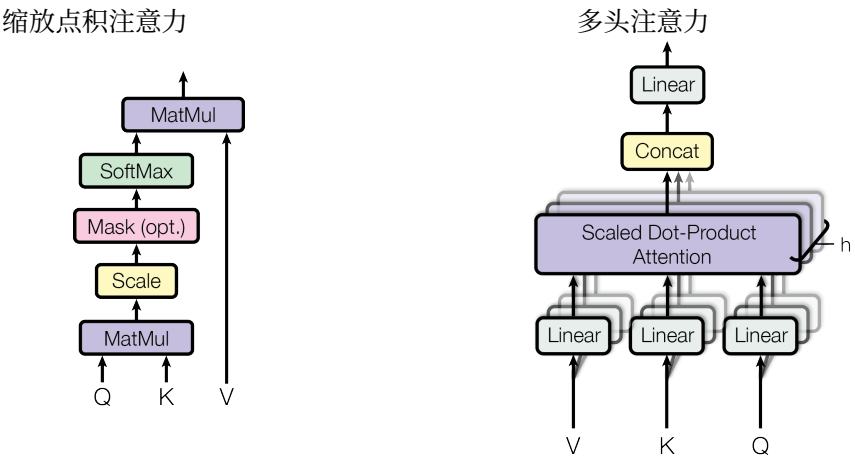


图2：（左）缩放点积注意力。（右）多头注意力由多个并行运行的注意力层组成。

值的加权和，其中分配给每个值的权重由查询与对应键的相容性函数计算得到。

3.2.1 缩放点积注意力

我们将所采用的注意力机制称为“缩放点积注意力”（图2）。输入包括维度为 d_k 的查询和键，以及维度为 d_v 的值。我们计算查询与所有键的点积，将每个点积除以 $\sqrt{d_k}$，再应用 softmax 函数，得到各个值的权重。

在实际计算中，我们将一组查询组成矩阵 Q，同时计算它们的注意力函数。键和值也分别组成矩阵 K 和 V。输出矩阵的计算方式为：

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V \tag{1}$$

最常用的两种注意力函数是加性注意力 [2]，和点积（乘性）注意力。除缩放因子 $\frac{1}{\sqrt{d_k}}$ 外，点积注意力与我们的算法相同。加性注意力使用具有单个隐藏层的前馈网络来计算相容性函数。尽管两者的理论复杂度相近，但在实际计算中，点积注意力速度更快、空间效率更高，因为它可以通过高度优化的矩阵乘法代码实现。

当 d_k 较小时，两种机制的表现相近；但当 d_k [3] 较大时，加性注意力优于未经缩放的点积注意力。我们推测，当 d_k 较大时，点积的量级也会增大，使 softmax 函数进入梯度极小的区域 [4]。为抵消这一影响，我们将点积乘以 $\frac{1}{\sqrt{d_k}}$ 进行缩放。

3.2.2 多头注意力

我们发现，与其对 d_{model} 维的键、值和查询执行一次注意力函数，不如使用不同的、通过学习得到的线性投影，将查询、键和值投影 h 次，分别投影到 d_k、d_k 和 d_v 维。随后，我们对每组投影后的查询、键和值并行执行注意力函数，得到 d_v 维的

[4]为说明点积为何会变大，假设 q 和 k 的各分量都是均值为0、方差为1的独立随机变量。那么，它们的点积 $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ 的均值为0、方差为 d_k。